BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Practical No: 1

Division: C Batch: 2

Roll no: 3090341 PRN No: 12410595

Name of Student: Pranav Khatod

Subject Name : Deep Learning

Problem Statement:

Implement a neural network using TensorFlow/Keras for handwritten digit classification on the MNIST dataset. Load and preprocess the data, define a Sequential model, train it, and evaluate test accuracy.

Algorithm / Methodology:

Libraries Used:

1. TensorFlow (tensorflow)

2. Matplotlib (matplotlib.pyplot)

Data Preparation

The MNIST dataset is loaded using tf.keras.datasets.mnist.load_data(). Pixel values are scaled from the range 0–255 to 0–1 by dividing by 255.0 so that training is numerically stable.

Training set shape is (60000, 28, 28) and test set shape is (10000, 28, 28). A sample digit image is visualized using matplotlib.

Model Architecture

Flatten Layer: Converts each 28×28 image into a 784-dimensional vector.

Hidden Dense Layer: 128 neurons with ReLU activation to learn non-linear features.

Output Dense Layer: 10 neurons with Softmax activation for digit classes 0–9.

Model Compilation and Training

The model is compiled with the Adam optimizer, sparse_categorical_crossentropy loss, and accuracy as the metric. It is trained for 3 epochs on the training images.

Evaluation

The trained model is evaluated on the unseen test set. The model is then saved as my_mnist_model.

Experiments and Results

Training accuracy improved across epochs: 0.9255 (epoch 1), 0.9672 (epoch 2), and 0.9773 (epoch 3). Final test accuracy obtained is 0.9763 with a test loss of 0.0777.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Get data and split into train and test datasets
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

# Scale down values of pixels from 0-255 to 0-1
train_images = train_images /255.0
test_images = test_images / 255.0

# Visualizing data
print(train_images.shape)
print(test_images.shape)
print(train_labels)

# Display first image
plt.imshow(train_images[1], cmap='gray')
plt.show()

# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))     #Rectified Linear Unit        #Dense layers are used to learn complex patterns in data.
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))   #softmax is often used as the activation function in the output layer for classification tasks

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
#Adaptive Moment Estimation #sparse_categorical_crossentropy for multi-class classification

# Training the model
```

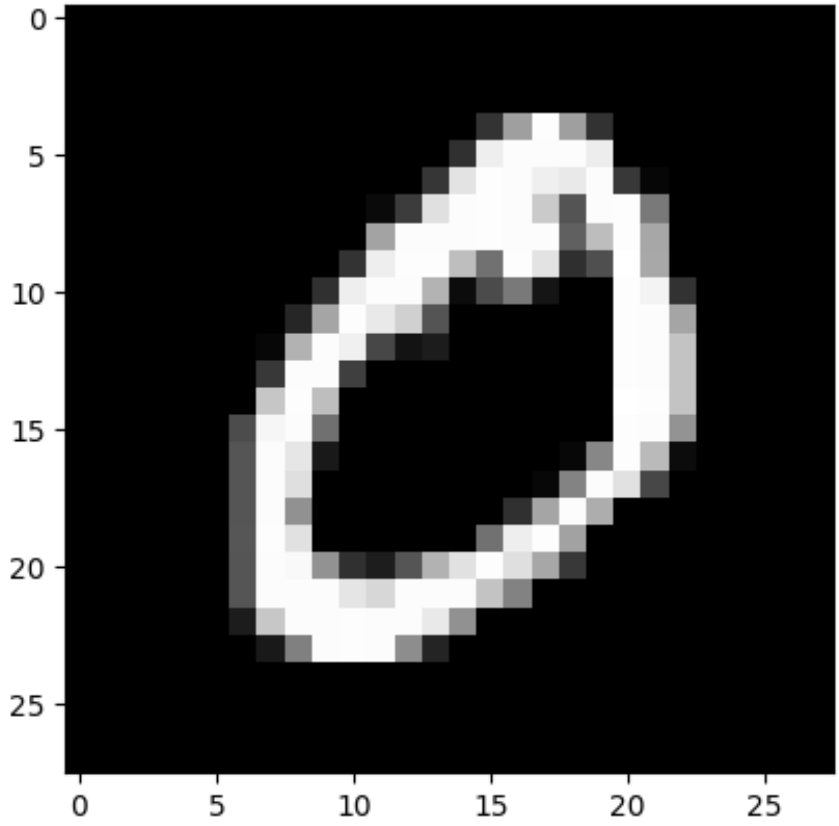
```
my_model.fit(train_images, train_labels, epochs=3)

# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

# Saving the model
my_model.save('my_mnist_model')
```

Output :

1. Visualization of MNIST sample image



2. Training logs

```
Epoch 1/3
1875/1875 - loss: 0.2617 - accuracy: 0.9255
Epoch 2/3
1875/1875 - loss: 0.1128 - accuracy: 0.9672
Epoch 3/3
1875/1875 - loss: 0.0762 - accuracy: 0.9773
```

3. Test evaluation

```
313/313 - loss: 0.0777 - accuracy: 0.9763
Test accuracy:  0.9763000011444092
```